

Semester	T.E. Semester V– BIOM
Subject	Biomedical Digital Signal Processing
Subject Professor In-charge	Prof. Geetha Narayanan
Student Name	Rehan Shaikh
Roll Number	24105A0034
Grade and Subject	
Teacher's Signature	

Week 5 Submission	
Project Title	PCG
Weekly Objective	DFT using Twiddle Factor FFT Comparison of FFT and DFT Add Powerline Salt and pepper
Key concepts learned	Salt and pepper DFT point
Work Completed	Week5 all work done
Code	<pre># ===== # PCG DFT USING TWIDDLE FACTOR # MIT-BIH Record 100 # =====</pre>

```
!pipinstall wfdb -q

import wfdb

import numpy as np

import matplotlib.pyplot as plt

# -----

# 1. Load PCG Record

# -----

record_path = "/content/100"

record = wfdb.rdrecord(record_path)

pcg = record.p_signal[:, 0] # Changed from ecg to pcg

fs = record.fs

print("Sampling Frequency:", fs, "Hz")
print("Total Samples:", len(pcg))
print("Channel:", record.sig_name[0])

# -----

# 2. Select 1000 samples

# -----

N = 1000
```

```
x=pcg[:N] # Changed from ecg to pcg
```

```
t= np.arange(N) / fs
```

```
#-----
```

```
#3. Plot Original PCG
```

```
#-----
```

```
plt.figure(figsize=(14,4))
```

```
plt.plot(t, x)
```

```
plt.xlabel("Time (seconds)")
```

```
plt.ylabel("Amplitude")
```

```
plt.title("Original PCG Signal")
```

```
plt.grid()
```

```
plt.show()
```

```
# -----
```

```
# 4. Twiddle Factor
```

```
# -----
```

```
W = np.exp(-2j * np.pi / N)
```

```
print("\nTwiddle Factor:")
```

```
print("W =", W)
```

```
# -----
```

```
# 5.DFT using Twiddle Factor

# -----

X = np.zeros(N, dtype=complex)

for k in range(N):

    for n in range(N):

        #Twiddle factor  $W^{kn}$ 

        twiddle = W ** (k * n)

        #DFT equation

        X[k] += x[n] * twiddle

print("\nDFT calculation completed.")

# -----

# 6. Magnitude Spectrum

# -----

magnitude = np.abs(X)

# Frequency axis

frequencies = np.arange(N) * fs / N
```

```
# -----  
  
# 7. Plot DFT  
  
# -----  
  
plt.figure(figsize=(14,5))  
  
plt.plot(  
    frequencies[:N//2],  
    magnitude[:N//2]  
)  
  
plt.xlabel("Frequency (Hz)")  
plt.ylabel("|X[k]|")  
plt.title("DFT Magnitude Spectrum using Twiddle Factor (PCG)")  
plt.grid()  
plt.show()  
  
//  
import numpy as np  
import matplotlib.pyplot as plt  
  
# The 'pcg' variable should be available from previous cells after executing  
'xSGnX8EgsXSK'.  
  
# The 'fs' (sampling frequency) variable should also be available.  
  
# Compute FFT of the entire PCG signal  
X = np.fft.fft(pcg)
```

```
# Number of samples
N = len(pcg)

# Generate frequency bins
freq = np.fft.fftfreq(N, d=1/fs)

# Select positive frequencies for plotting
positive_frequencies_mask = freq >= 0
freq_positive = freq[positive_frequencies_mask]
magnitude_positive = np.abs(X)[positive_frequencies_mask]
phase_positive = np.angle(X)[positive_frequencies_mask]

# Plot Magnitude Spectrum
plt.figure(figsize=(15, 6))
plt.plot(freq_positive, magnitude_positive)
plt.title('Magnitude Spectrum of PCG Signal (FFT)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.grid(True)
plt.xlim(0, fs/2) # Limit x-axis to Nyquist frequency
plt.show()

# Plot Phase Spectrum
plt.figure(figsize=(15, 6))
plt.plot(freq_positive, phase_positive)
```

```
plt.title('Phase Spectrum of PCG Signal (FFT)')

plt.xlabel('Frequency (Hz)')

plt.ylabel('Phase (radians)')

plt.grid(True)

plt.xlim(0, fs/2) # Limit x-axis to Nyquist frequency

plt.show()


//

print(f"Total magnitude values for 1000 samples: {len(magnitude)}")

print("First 20 magnitude values:")

foriin range(min(20, len(magnitude))):

    print(f"Magnitude[{i}] = {magnitude[i]:.4f}")


//

#The 'magnitude' variable already holds the magnitude spectrum

#derived from the FFT of the first 1000 PCG samples.

#The 'freq' variable (or 'frequencies' from cell xSGnX8EgsXSK) holds the

corresponding frequencies.


# Assuming 'frequencies' and 'magnitude' from cell `fjPsVU6r10Dx` are still

available in the kernel state.

# If not, these variables would need to be re-computed from `pcg[:1000]` .


plt.figure(figsize=(14, 5))

plt.plot(

    frequencies[:N//2],

    magnitude[:N//2]
```

```
)  
  
plt.xlabel("Frequency (Hz)")  
  
plt.ylabel("Magnitude")  
  
plt.title("FFT Magnitude Spectrum of First 1000 PCG Samples")  
  
plt.grid(True)  
  
plt.show()  
  
  
//  
  
# The 'magnitude' variable already holds the magnitude spectrum  
# derived from the FFT of the first 1000 PCG samples.  
  
# The 'freq' variable (or 'frequencies' from cell xSGnX8EgsXSK) holds the  
# corresponding frequencies.  
  
  
  
# Assuming 'frequencies' and 'magnitude' from cell `fjPsVU6r10Dx` are still  
# available in the kernel state.  
  
# If not, these variables would need to be re-computed from `pcg[:1000]` .  
  
  
  
plt.figure(figsize=(14, 5))  
  
plt.plot(  
    frequencies[:N//2],  
    magnitude[:N//2]  
)  
  
plt.xlabel("Frequency (Hz)")  
  
plt.ylabel("Magnitude")  
  
plt.title("FFT Magnitude Spectrum of First 1000 PCG Samples")  
  
plt.grid(True)  
  
plt.show()
```



```
//  
  
import time  
  
# Ensure x and N are consistently set for the comparison  
x = pcg[:100] # Use the first 100 samples of pcg  
N = len(x) # N is now 100  
  
print(f"Comparing DFT and FFT for {N} samples.\n")  
  
# --- Custom DFT Implementation ---  
  
start_time_dft = time.time()  
X_dft = np.zeros(N, dtype=complex)  
  
for k_dft in range(N):  
    for n_dft in range(N):  
        X_dft[k_dft] += x[n_dft] * np.exp(-2j * np.pi * n_dft * k_dft / N)  
  
end_time_dft = time.time()  
time_dft = end_time_dft - start_time_dft  
print(f"Time taken for custom DFT: {time_dft:.6f} seconds")  
  
# --- NumPy's FFT Implementation ---  
  
start_time_fft = time.time()  
X_fft = np.fft.fft(x)
```

```
end_time_fft = time.time()

time_fft = end_time_fft - start_time_fft

print(f"Time taken for NumPy's FFT: {time_fft:.6f} seconds")

print(f"\nNumPy's FFT is {time_dft/time_fft:.2f} times faster than custom DFT  
for {N} samples.")

//

import numpy as np

import matplotlib.pyplot as plt

# Ensure 'x' (first 100 pcg samples) and 'fs' (sampling frequency) are  
available.

# 'x' is currently pcg[:100] and 'fs' is 360 from previous cells.

N_100_samples = len(x)

fft_values_100_samples = np.fft.fft(x)

freq_100_samples = np.fft.fftfreq(N_100_samples, d=1/fs)

positive_freq_mask = freq_100_samples >= 0

plt.figure(figsize=(10,5))

plt.stem(freq_100_samples[positive_freq_mask],  
np.abs(fft_values_100_samples)[positive_freq_mask])
plt.title("FFT Magnitude Spectrum of First 100 PCG Samples")

plt.xlabel("Frequency (Hz)")

plt.ylabel("Magnitude")

plt.grid(True)

plt.show()
```

```
//  
  
import time  
  
import numpy as np  
  
  
# Ensure 'pcg' is available from earlier record loading  
# Ensure x and N are consistently set for the comparison  
x = pcg[:100] # Use the first 100 samples of pcg  
N = len(x) # N is now 100  
  
  
print(f"Comparing DFT and FFT for {N} samples.\n")  
  
  
# --- Custom DFT Implementation ---  
  
  
start_time_dft = time.time()  
X_dft = np.zeros(N, dtype=complex)  
  
  
for k_dft in range(N):  
    for n_dft in range(N):  
        X_dft[k_dft] += x[n_dft] * np.exp(-2j * np.pi * n_dft * k_dft / N)  
  
  
end_time_dft = time.time()  
time_dft = end_time_dft - start_time_dft  
print(f"Time taken for custom DFT: {time_dft:.6f} seconds")  
  
  
# --- NumPy's FFT Implementation ---
```

```
start_time_fft = time.time()

X_fft = np.fft.fft(x)

end_time_fft = time.time()

time_fft = end_time_fft - start_time_fft

print(f"Time taken for NumPy's FFT: {time_fft:.6f} seconds")


print(f"\nNumPy's FFT is {time_dft/time_fft:.2f} times faster than custom DFT
for {N} samples.")

//

import numpy as np

import matplotlib.pyplot as plt


# Compute FFT of the entire PCG signal (variable 'pcg' should be available)
X_all = np.fft.fft(pcg)


# Extract the magnitude spectrum
magnitude_all = np.abs(X_all)


# Get the first 5000 magnitude values
magnitude_5000 = magnitude_all[:1000]


print(f"Displaying the magnitude of the first {len(magnitude_5000)} frequency
components:")

for i, mag in enumerate(magnitude_5000):
    print(f"Component {i}: {mag:.4f}")


#Ensure 'freq_positive' is available from previous FFT calculations (e.g., from
cell '8e5762db')
```

```
# Limit to the first 5000 frequency components

num_components_to_plot = 5000

# Make sure we don't try to plot more components than available
plot_freq = freq_positive[:min(len(freq_positive), num_components_to_plot)]

plot_magnitude = magnitude_all[:min(len(magnitude_all),
num_components_to_plot)]

plt.figure(figsize=(15, 6))
plt.plot(plot_freq, plot_magnitude)

plt.title(f'Magnitude Spectrum of First {len(plot_magnitude)} Frequency
Components')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')

plt.grid(True)

plt.show()

//

component_index = 4999 # For the 5000th component (0-indexed)

if component_index < len(freq_positive) and component_index <
len(magnitude_all):

    frequency_5000th = freq_positive[component_index]

    magnitude_5000th = magnitude_all[component_index]

    print(f"Frequency of the {component_index + 1}th component:
{frequency_5000th:.4f} Hz")

    print(f"Magnitude of the {component_index + 1}th component:
{magnitude_5000th:.4f}")

else:
```

```
print(f"Error: The {component_index + 1}th component is out of bounds.  
The signal has fewer frequency components.")  
  
//  
  
import matplotlib.pyplot as plt  
  
# Ensure time_dft and time_fft are available from the comparison cell (e.g.,  
'29d8187a')  
  
labels = ['Custom DFT', 'NumPy FFT']  
times = [time_dft, time_fft]  
  
plt.figure(figsize=(8, 5))  
plt.bar(labels, times, color=['skyblue', 'lightcoral'])  
plt.title('Computation Time Comparison: Custom DFT vs. NumPy FFT (100  
Samples)')  
plt.xlabel('Method')  
plt.ylabel('Time (seconds)')  
plt.grid(axis='y', linestyle='--', alpha=0.7)  
plt.show()  
  
//  
  
import time  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
# Number of samples  
  
N = 1000
```

#ECG signal

x= ecg[:N]

#=====

#1. FFT TIME

#=====

start_fft = time.perf_counter()

X_fft = np.fft.fft(x)

end_fft = time.perf_counter()

fft_time = end_fft - start_fft

print("FFT Computation Time =", fft_time, "seconds")

#=====

2. DFT USING TWIDDLE FACTOR

#=====

start_dft = time.perf_counter()

X_dft = np.zeros(N, dtype=complex)

W = np.exp(-2j * np.pi / N)

```

forkin range(N):
    forn in range(N):
        X_dft[k] += x[n] * (W ** (k * n))

end_dft = time.perf_counter()

dft_time = end_dft - start_dft

print("DFT Computation Time =", dft_time, "seconds")

# =====
# 3. Comparison
# =====

print("\n=====")
print("COMPUTATION TIME COMPARISON")
print("=====")

print("FFT :", fft_time, "seconds")
print("DFT :", dft_time, "seconds")

print("\nDFT is approximately",
      dft_time / fft_time,
      "times slower than FFT.")

```



```
//
# =====

# ADD 50 Hz POWER-LINE NOISE TO ECG SIGNAL
# =====

import numpy as np
import matplotlib.pyplot as plt

# Number of samples
N = 1000

#Original ECG signal
x=ecg[:N]

#Sampling frequency
fs=360

#Time vector
t= np.arange(N) / fs

#-----
#Power-line noise parameters
#-----

noise_frequency = 50 # 50 Hz power-line interference
noise_amplitude = 0.1 # Adjust noise strength
```

```
# Generate 50 Hz sinusoidal noise

powerline_noise = noise_amplitude * np.sin(

    2 * np.pi * noise_frequency * t
)

#-----

#Add noise to ECG

#-----

noisy_ecg = x + powerline_noise

# -----

# Plot original and noisy ECG

# -----

plt.figure(figsize=(14, 8))

plt.subplot(2, 1, 1)

plt.plot(t, x)

plt.xlabel("Time (seconds)")

plt.ylabel("Amplitude")

plt.title("Original PCG Signal")

plt.grid()

plt.subplot(2, 1, 2)
```

```
plt.plot(t, noisy_ecg)

plt.xlabel("Time (seconds)")

plt.ylabel("Amplitude")

plt.title("PCG Signal with 50 Hz Power-Line Noise")

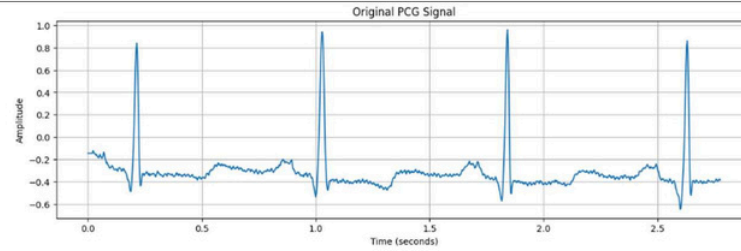
plt.grid()


plt.tight_layout()

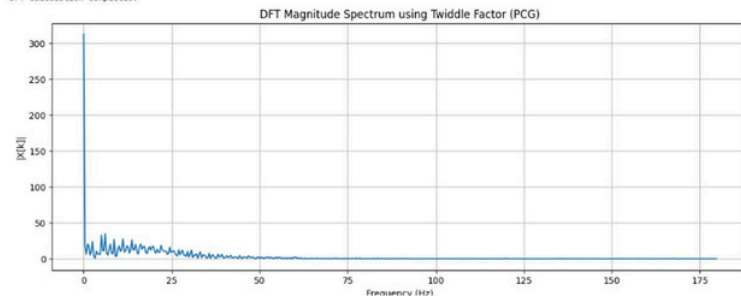
plt.show()

//
```

Evidence of
Progress/Res
ults



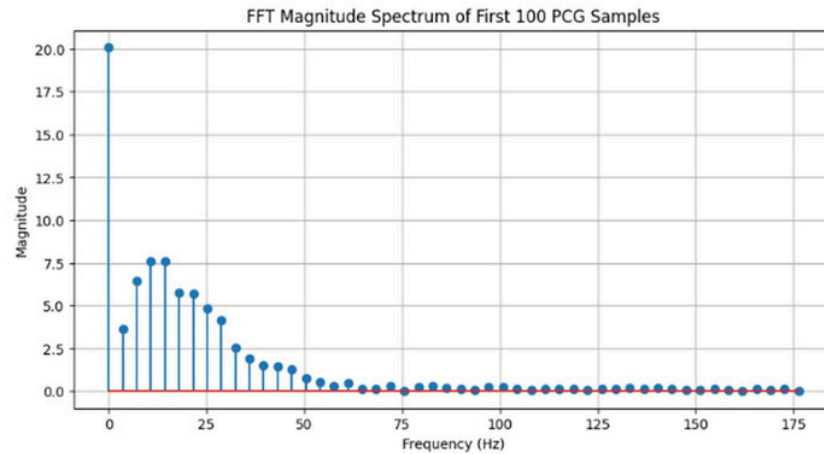
Twiddle factor:
 $W = (0.999902608561371 - 0.00628314396558951j)$
DFT calculation completed.

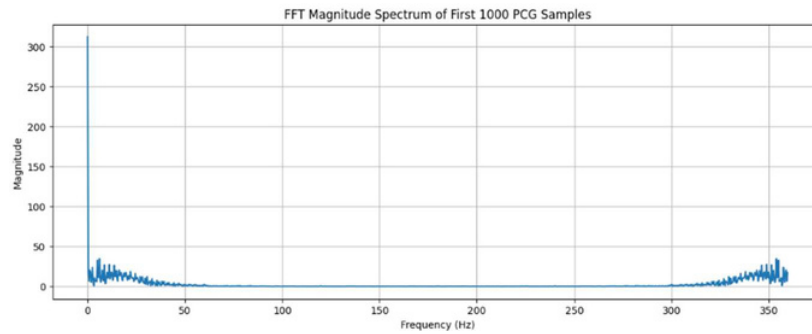
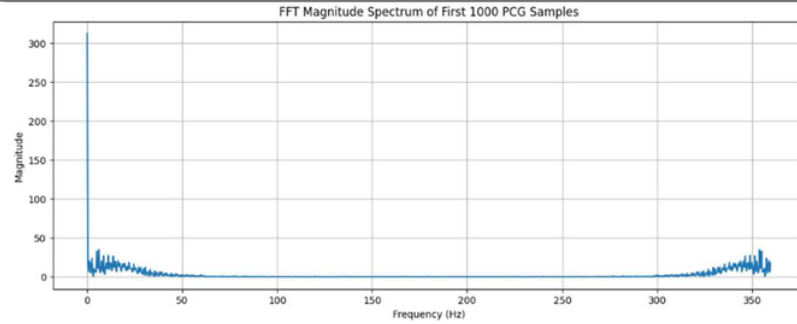


Total magnitude values for 1000 samples: 1000

First 20 magnitude values:

Magnitude[0] = 311.8850
Magnitude[1] = 17.3882
Magnitude[2] = 6.5024
Magnitude[3] = 20.5210
Magnitude[4] = 18.3879
Magnitude[5] = 5.3870
Magnitude[6] = 9.9478
Magnitude[7] = 23.8288
Magnitude[8] = 3.8734
Magnitude[9] = 0.9011
Magnitude[10] = 10.7244
Magnitude[11] = 6.5223
Magnitude[12] = 6.8251
Magnitude[13] = 5.9606
Magnitude[14] = 32.8464
Magnitude[15] = 11.7535
Magnitude[16] = 11.0416
Magnitude[17] = 34.6817
Magnitude[18] = 8.6685
Magnitude[19] = 5.2261





Comparing DFT and FFT for 100 samples.

Time taken for custom DFT: 0.047251 seconds

Time taken for NumPy's FFT: 0.000893 seconds

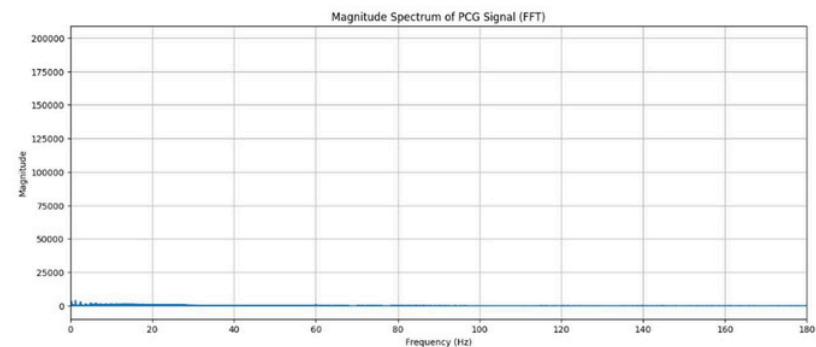
NumPy's FFT is 52.91 times faster than custom DFT for 100 samples.

Comparing DFT and FFT for 100 samples.

Time taken for custom DFT: 0.055595 seconds

Time taken for NumPy's FFT: 0.00332 seconds

NumPy's FFT is 167.28 times faster than custom DFT for 100 samples.



Dominant Frequency: 1.25 Hz
Corresponding Magnitude: 199969

```

** Frequency of the 5000th component: 2.7687 Hz
   Magnitude of the 5000th component: 585.7222

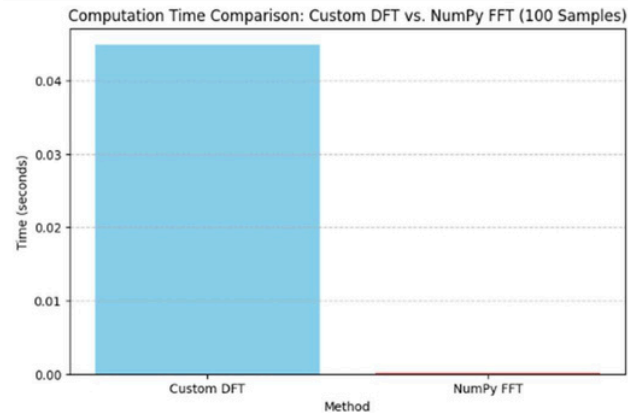
FFT Computation Time = 0.0003469639996183105 seconds
DFT Computation Time = 2.7944008410004244 seconds

=====
COMPUTATION TIME COMPARISON
=====
FFT : 0.0003469639996183105 seconds
DFT : 2.7944008410004244 seconds

DFT is approximately 8053.863928460877 times slower than FFT.

... Displaying the magnitude of the first 1000 frequency components:
Component 0: 199094.3350
Component 1: 3380.3149
Component 2: 817.6116
Component 3: 78.7525
Component 4: 2547.3783
Component 5: 1497.2152
Component 6: 890.4243
Component 7: 1795.3887
Component 8: 3109.4799
Component 9: 1296.4838
Component 10: 1313.8400
Component 11: 1566.9593
Component 12: 1100.4206
Component 13: 1713.9142
Component 14: 2211.4310
Component 15: 3646.6339
Component 16: 544.0987
Component 17: 1778.7046
Component 18: 1593.1593
Component 19: 1397.0068
Component 20: 1711.0554
Component 21: 2212.5122
Component 22: 673.4957
Component 23: 2338.2406
Component 24: 1766.6688
Component 25: 755.6688
Component 26: 1818.4980
Component 27: 2073.0162
Component 28: 221.6351
Component 29: 1350.7047
Component 30: 1078.7633
Component 31: 1092.2517
Component 32: 446.4040
Component 33: 1259.1499
Component 34: 391.8906
Component 35: 1606.1890
Component 36: 1524.3890
Component 37: 615.3418
Component 38: 3116.5020
Component 39: 1025.4806
Component 40: 829.1315
Component 41: 1247.6225
Component 42: 941.1788
Component 43: 931.1496
Component 44: 2104.3347
Component 45: 241.4678
Component 46: 585.7380
Component 47: 350.9302
Component 48: 513.5421
Component 49: 1688.3878
Component 50: 616.4056
Component 51: 294.1522
Component 52: 1657.7317
Component 53: 1066.9556
Component 54: 638.6400
Component 55: 737.9944

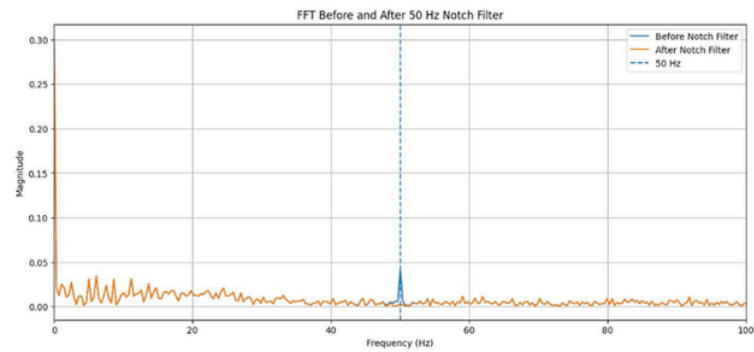
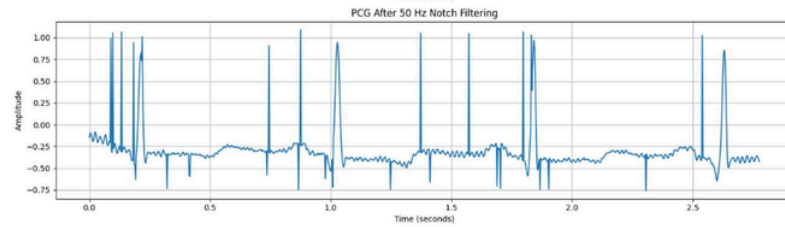
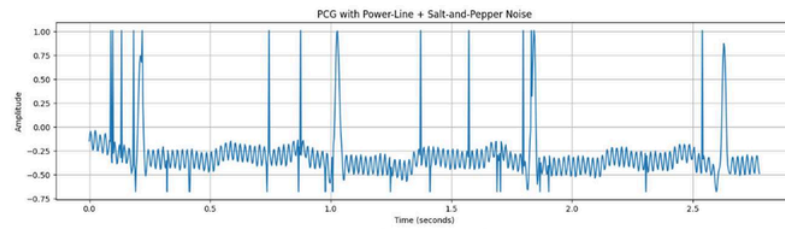
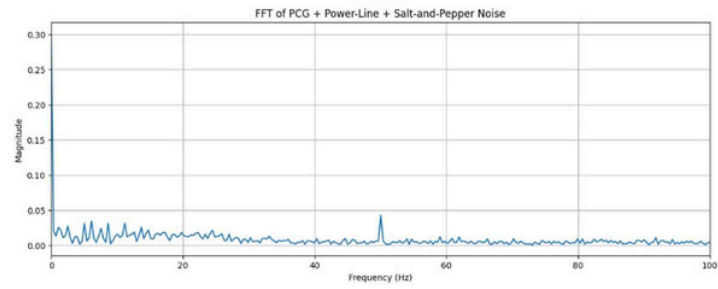
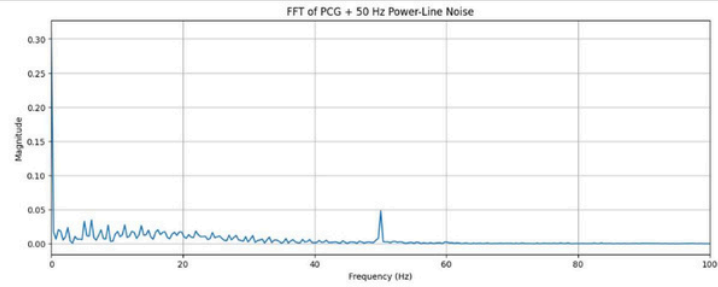
```

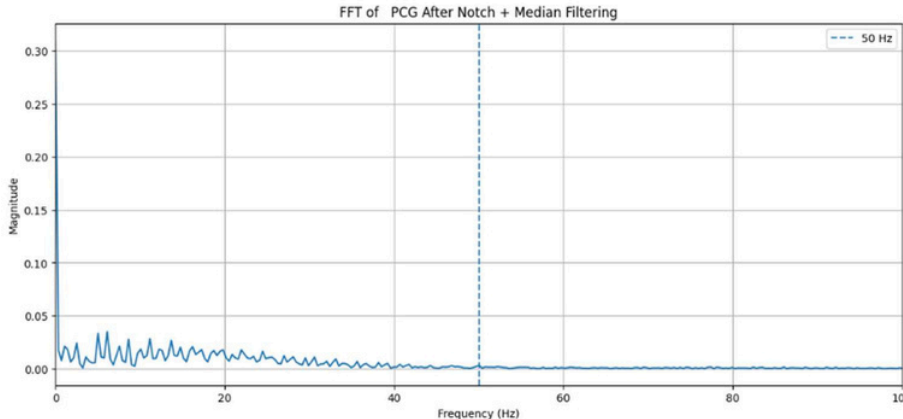
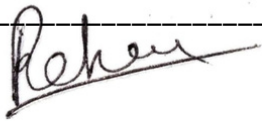


*** Frequency Resolution = 0.36 Hz

Significant Frequency Components:

```
-----
Frequency = 0.00 Hz, Magnitude = 0.311885
Frequency = 0.36 Hz, Magnitude = 0.017388
Frequency = 1.08 Hz, Magnitude = 0.020521
Frequency = 1.44 Hz, Magnitude = 0.018388
Frequency = 2.52 Hz, Magnitude = 0.023829
Frequency = 5.04 Hz, Magnitude = 0.032846
Frequency = 6.12 Hz, Magnitude = 0.034682
Frequency = 7.56 Hz, Magnitude = 0.020295
Frequency = 8.64 Hz, Magnitude = 0.026969
Frequency = 10.08 Hz, Magnitude = 0.017997
Frequency = 11.16 Hz, Magnitude = 0.027447
Frequency = 12.24 Hz, Magnitude = 0.018201
Frequency = 12.60 Hz, Magnitude = 0.016442
Frequency = 13.68 Hz, Magnitude = 0.026339
Frequency = 14.76 Hz, Magnitude = 0.019886
Frequency = 15.84 Hz, Magnitude = 0.016872
Frequency = 16.20 Hz, Magnitude = 0.020412
Frequency = 16.92 Hz, Magnitude = 0.016667
Frequency = 17.28 Hz, Magnitude = 0.017446
Frequency = 18.72 Hz, Magnitude = 0.016751
Frequency = 19.44 Hz, Magnitude = 0.017155
Frequency = 19.80 Hz, Magnitude = 0.017242
Frequency = 21.96 Hz, Magnitude = 0.018609
Frequency = 22.32 Hz, Magnitude = 0.018275
-----
```



	
Challenges & Solutions	We faced many errors in this week and get solution by using chat gpt
Next Week Plan	Week 6 work
Declaraion	I hereby declare that for the assignment / academic activity submitted by me, I confirm ONE of the following statements by ticking (☐) the appropriate box: ☐ (1) I have prepared the assignment / academic activity entirely by myself. ☐ (2) I have completed the assignment / academic activity with academic guidance or discussion from seniors / friends / peers, while ensuring the work is my own. ☐ (3) I have used AI-based tools only as an aid and have appropriately acknowledged or cited their use as per the ethical guidelines of the Institute. ☐ (4) I have used AI-based tools and directly copy-pasted content for the assignment / academic activity. ☐ (5) I have copied the assignment / academic activity from peers / friends. I understand that selecting options (4) or (5) indicates unethical academic practice and may attract academic penalties, including rejection of the assignment or disciplinary action, as per the rules of Vidyalankar Institute of Technology, Mumbai. I submit this declaration truthfully and accept full responsibility for the same. Student Name: Rehan Shaikh Roll No.: 24105A0034 Signature:  Date: _____